Diagrama de clases del proyecto de software. GA4-220501095-AA2-EV04.

Instructor: Libardo Gómez Diaz

Aprendiz: Juan Camilo Herrera Castellanos

Ficha: 3336085

SENA

Servicio Aprendizaje Nacional

2026



Índice

1. Introducción

2. Tipos de Diagramas de Clases

3. Diagrama de Clases Apuestas.

4. Descripción e Interpretación

5. Objetivos

6. Referencias Bibliográficas

Introducción.

El Lenguaje de Modelado Unificado (UML) es el estándar de la industria del software utilizado para especificar, visualizar y documentar la arquitectura de un sistema mediante representaciones gráficas estáticas y dinámicas. En el desarrollo de software, la importancia de estructurar el sistema a través de un diagrama de clases radica en que permite definir los objetos, atributos, métodos y relaciones lógicas antes de iniciar la fase de codificación. Este proceso de abstracción previa minimiza los errores conceptuales, optimiza la organización de las bases de datos y sirve como una guía clara para los desarrolladores, garantizando que el diseño del software responda de manera eficiente a los requisitos del negocio, como en este caso, la gestión de usuarios, cuentas y flujos financieros dentro de una plataforma de apuestas.

Tipos de Diagramas de Clases

diagrama de clases conceptual.

se enfoca en el análisis del negocio, representando las entidades del mundo real y sus conceptos abstractos sin definir detalles técnicos ni lenguajes de programación.

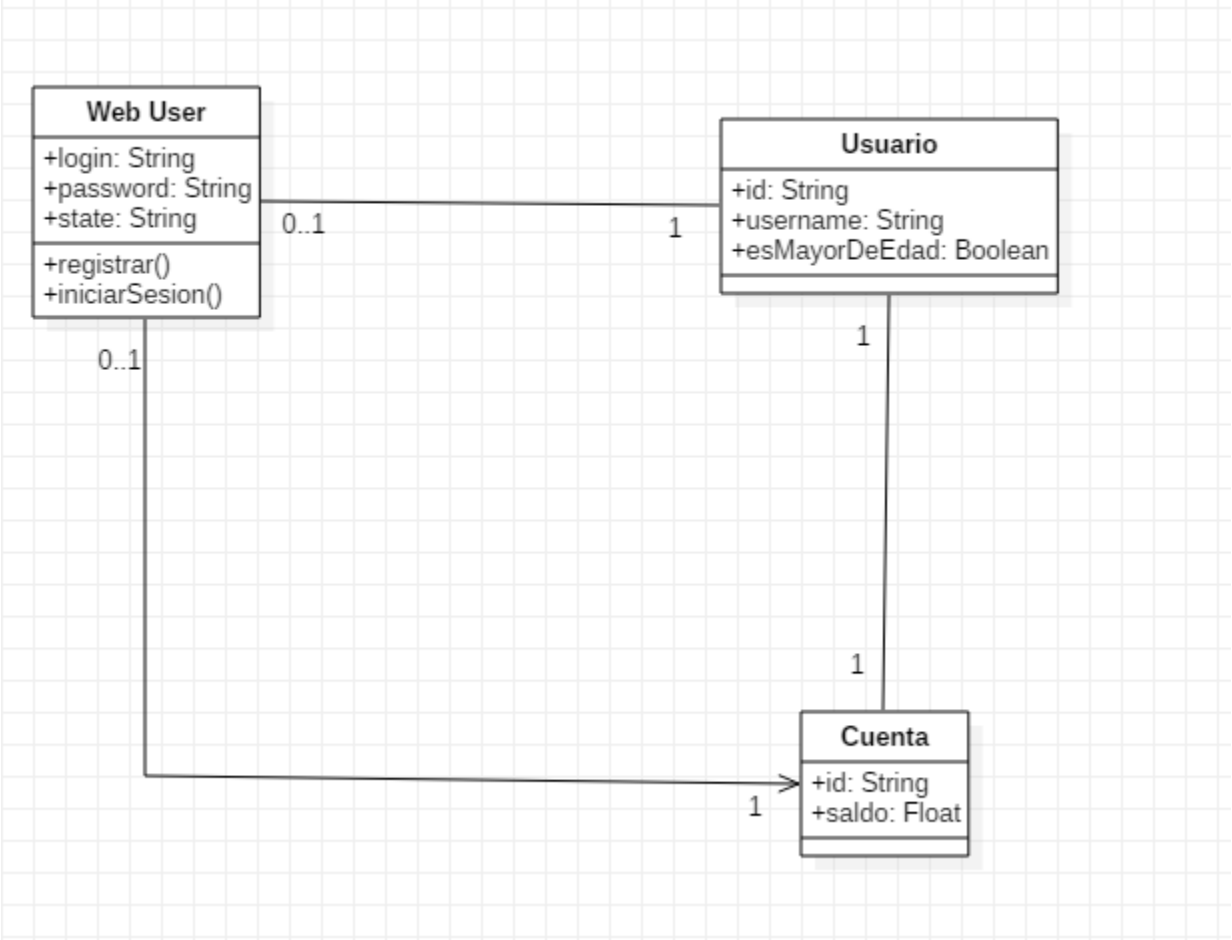
diagrama de clases de especificación.

introduce mayor formalidad técnica al detallar los tipos de datos de los atributos (String, Float, Boolean) y la visibilidad de los métodos, sirviendo como un puente directo hacia el diseño lógico del software.

diagrama de clases de implementación

reduce este diseño al entorno real de desarrollo, adaptando la arquitectura física a las restricciones de un lenguaje de programación específico (como Java o C#) o a un mapeo relacional de base de datos (ORM), incluyendo clases de persistencia, interfaces y controladores del backend.

Diagrama de Clases Apuestas.



Descripción e Interpretación

El diagrama de clases presentado modela el núcleo estático de autenticación, control de identidad y gestión financiera del sistema de apuestas. En primer lugar, la clase Web User representa el componente de acceso digital al sistema, encapsulando los atributos de seguridad (login, password, state) y las operaciones esenciales de autenticación (registrar(), iniciarSesion()). Esta clase se vincula mediante una relación de asociación bidimensional con la

clase Usuario con una multiplicidad de 1 a 0..1; esto establece la regla de negocio crítica de que un usuario humano real (identificado legalmente por su id, username y el validador esMayorDeEdad) solo puede poseer una única credencial de acceso activa en la web, mitigando fraudes por suplantación. A su vez, la clase Usuario se conecta de forma estricta (relación uno a uno 1 a 1) con la clase Cuenta, lo que garantiza que todo ciudadano registrado disponga automáticamente de un monedero financiero único para la persistencia de su capital. Finalmente, el diagrama incorpora una Asociación Dirigida (Directed Association) desde Web User hacia Cuenta con multiplicidad 0..1 a 1. Esta línea con flecha modela la navegabilidad del software en la capa de presentación: permite que el módulo de sesión de internet consulte y lea directamente los atributos financieros (como el saldo: Float) para desplegarlos en tiempo real en la interfaz del cliente, manteniendo un desacoplamiento donde el monedero contable no depende ni conoce la lógica del inicio de sesión web.

Objetivos

Objetivo General

Modelar de manera profunda y rigurosa la arquitectura estática del backend para una plataforma tecnológica de apuestas, estructurando mediante la metodología del Lenguaje de Modelado Unificado (UML) las entidades lógicas, sus reglas de negocio y los flujos financieros indispensables para garantizar la consistencia de los datos antes de la fase de codificación.

Objetivos Específicos

Identificar y abstraer las entidades del dominio: Definir las clases fundamentales del sistema (como Web User, Usuario, Cuenta, Pago y Apuesta), determinando con precisión sus

atributos intrínsecos y los tipos de datos requeridos para su posterior implementación en lenguajes orientados a objetos (POO) [Studocu].

Establecer las restricciones y lógica del negocio: Estructurar las relaciones y dependencias del sistema mediante el uso correcto de multiplicidades (conexiones uno a uno, uno a muchos) y navegabilidad [Studocu]. Esto con el fin de modelar restricciones críticas, como impedir que un usuario posea múltiples identidades web activas o que consulte saldos ajenos.

Optimizar el diseño relacional y financiero: Validar el comportamiento estático de los flujos de dinero y transacciones bancarias, asegurando que la persistencia del saldo y el registro de pagos sigan una estructura secuencial lógica que elimine errores lógicos de tipo contable en la base de datos [Studocu].

Facilitar la transición hacia el desarrollo técnico: Proveer una documentación técnica visual estandarizada en StarUML que sirva como plano de ingeniería de software, permitiendo la generación automática de código limpio o scripts SQL para acelerar el ciclo de vida del proyecto [Studocu].

Referencias Bibliograficas

Ambler, S. W. (2004). *The Elements of UML 2.0 Style* [El estilo de los elementos de UML 2.0]. Cambridge University Press.

Larman, C. (2003). *Aplicando UML y patrones: Una introducción al análisis y diseño orientado a objetos y al proceso unificado* (2ª ed.). Pearson Educación.

Mora, M. (2025). *Guía de aprendizaje ADSO: Análisis y desarrollo de software*. Servicio Nacional de Aprendizaje (SENA). sena.edu.co

Rumbaugh, J., Jacobson, I., & Booch, G. (2007). *El lenguaje de modelado unificado UML: Manual de referencia* (2ª ed.). Addison-Wesley.

StarUML. (2026). *StarUML 6 User Manual* [Manual de usuario de StarUML 6]. MKLab. staruml.io